

ACN Programming Assignment 2

Socket Programming: TCP and UDP Mini File Transfer

Roll Number 1: CS26MTECH11008
Name 1: SAI CHAITANYA MUPPARAJU

Roll Number 2: CS26MTECH11006
Name 2: GAJJALA JYOTHI SAI HARI TEJA

Department of Computer Science and Engineering
IIT Hyderabad

Date: 31-08-2026

Part 1 — TCP File Transfer

This part implements a simple FTP-style tool over TCP. A client connects to a server, and can list files on the server, upload a file with put, or download a file with get. Because TCP keeps a connection open and guarantees ordered, lossless delivery, the code does not need to handle missing or out-of-order data itself — the transport layer already takes care of that. To still be sure that a file arrives unchanged, every transfer is checked with a SHA-256 checksum on both ends.

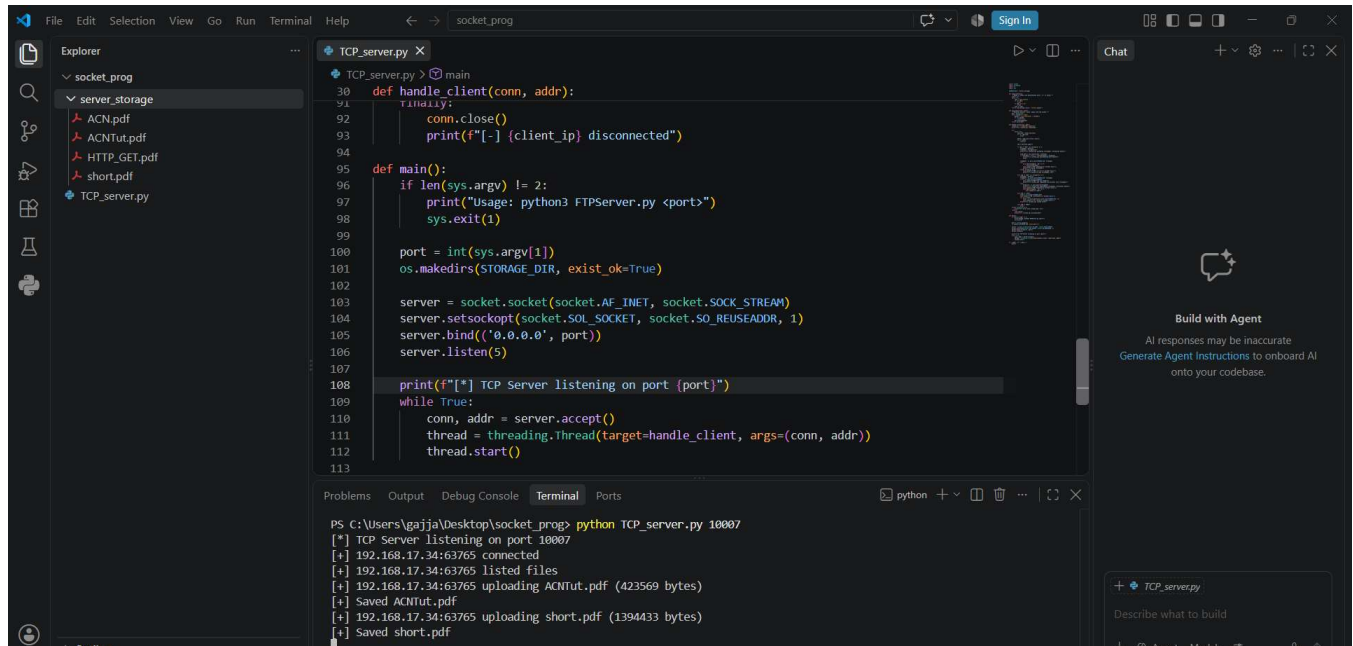
Uploading and downloading a file

The client first connects to the server and lists what's already stored there. A put command then sends a file across the server writes it to its storage folder and confirms with a save message. A get command does the reverse, the client asks for a named file and the server streams it back.

TCPFTP Client: PUT

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
[✓] Connected to the FTP server at 192.168.17.34:10007
ftp> list
Server contains [3] files:
- ACN.pdf 1394433
- ACNTut.pdf 423569
- HTTP_GET.pdf 119428
ftp> put ACNTut.pdf
[🔒] Calculating local file checksum...
SHA-256: 9c30ce997ef02a565e591c2d67569b34002b186b5e239bf91e94b55463afb48f
[🕒] Uploading ACNTut.pdf (423569 bytes)...
[✓] Success: Transferred 423569 total bytes cleanly.
ftp> put short.pdf
[🔒] Calculating local file checksum...
SHA-256: e95af85bb0af1392adb6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
[🕒] Uploading short.pdf (1394433 bytes)...
[✓] Success: Transferred 1394433 total bytes cleanly.
ftp> |
```

TCPFTP Server:



```
def handle_client(conn, addr):
    try:
        conn.close()
        print(f"[-] {client_ip} disconnected")
    except:
        pass

def main():
    if len(sys.argv) != 2:
        print("Usage: python3 FTPServer.py <port>")
        sys.exit(1)

    port = int(sys.argv[1])
    os.makedirs(STORAGE_DIR, exist_ok=True)

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(('0.0.0.0', port))
    server.listen(5)

    print(f"[*] TCP Server listening on port {port}")
    while True:
        conn, addr = server.accept()
        thread = threading.Thread(target=handle_client, args=(conn, addr))
        thread.start()

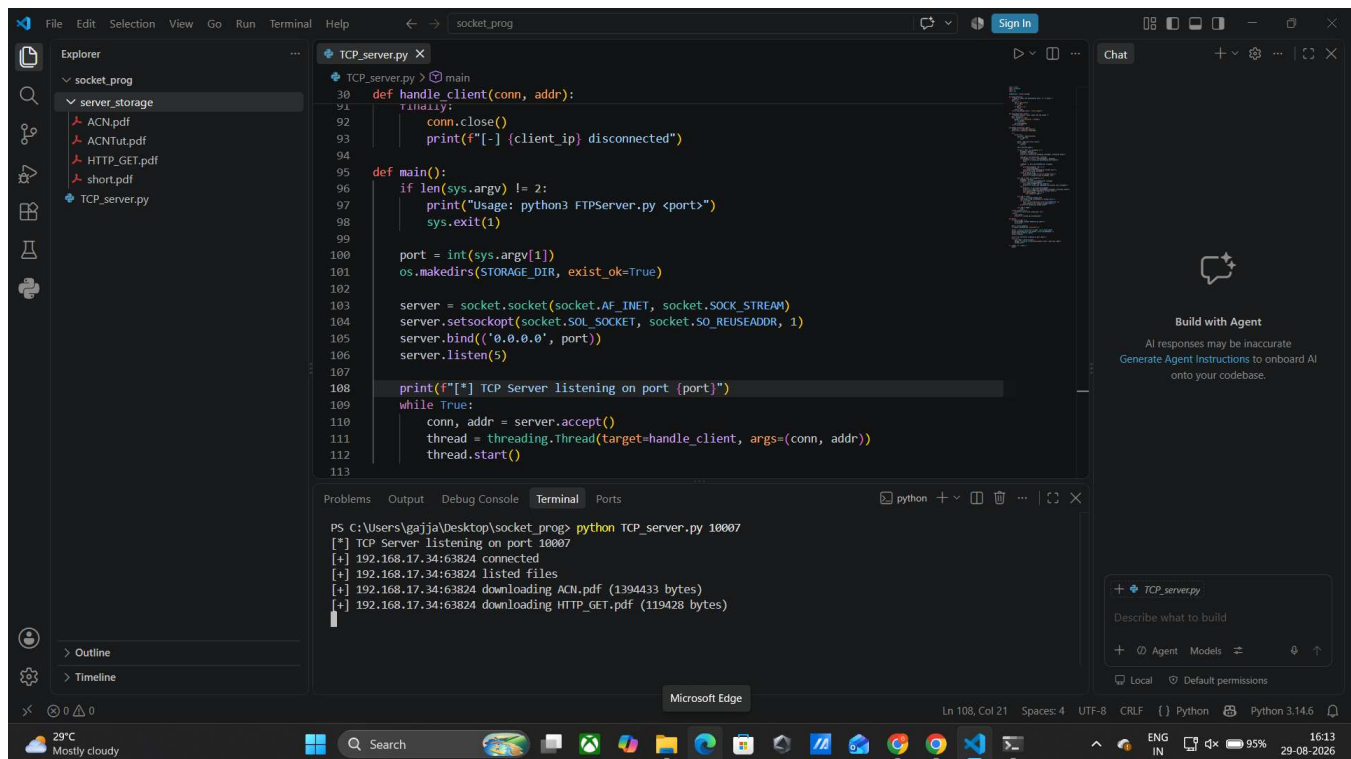
if __name__ == '__main__':
    main()
```

```
PS C:\Users\gajja\Desktop\socket_prog> python TCP_server.py 10007
[*] TCP Server listening on port 10007
[+] 192.168.17.34:63765 connected
[+] 192.168.17.34:63765 listed files
[+] 192.168.17.34:63765 uploading ACNTut.pdf (423569 bytes)
[+] Saved ACNTut.pdf
[+] 192.168.17.34:63765 uploading short.pdf (1394433 bytes)
[+] Saved short.pdf
```

TCPFTP CLIENT GET:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
[✓] Connected to the FTP server at 192.168.17.34:10007
ftp> list
  Server contains [4] files:
  - ACN.pdf 1394433
  - ACNTut.pdf 423569
  - HTTP_GET.pdf 119428
  - short.pdf 1394433
ftp> get ACN.pdf
⚠Note: Local file already exists. Auto-renamed destination target to: ACN_1.pdf
[✓] Success: Downloaded 1394433 bytes to ACN_1.pdf.
🔒 Verifying integrity checksum of downloaded file...
  SHA-256: e95af85bb0af1392adb6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
ftp> get HTTP_GET.pdf
⚠Note: Local file already exists. Auto-renamed destination target to: HTTP_GET_1.pdf
[✓] Success: Downloaded 119428 bytes to HTTP_GET_1.pdf.
🔒 Verifying integrity checksum of downloaded file...
  SHA-256: 6d3e72dc5244c229c7aa69a946799be4aa709ea75f4f95710464c72a8e5e7fea
```

TCPFTP Server :



```
socket_prog
├── server_storage
│   ├── ACN.pdf
│   ├── ACNTut.pdf
│   ├── HTTP_GET.pdf
│   ├── short.pdf
│   └── TCP_server.py
└── TCP_server.py X
    30 def handle_client(conn, addr):
    31     try:
    32         conn.close()
    33         print(f"[-] {client_ip} disconnected")
    34     except:
    35         pass
    36
    37 def main():
    38     if len(sys.argv) != 2:
    39         print("Usage: python3 FTPServer.py <port>")
    40         sys.exit(1)
    41
    42     port = int(sys.argv[1])
    43     os.makedirs(STORAGE_DIR, exist_ok=True)
    44
    45     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    46     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    47     server.bind(('0.0.0.0', port))
    48     server.listen(5)
    49
    50     print(f"[*] TCP Server listening on port {port}")
    51     while True:
    52         conn, addr = server.accept()
    53         thread = threading.Thread(target=handle_client, args=(conn, addr))
    54         thread.start()
    55
    56 if __name__ == '__main__':
    57     main()
    58
    59 Problems Output Debug Console Terminal Ports
    60
    61 PS C:\Users\gajja\Desktop\socket_prog> python TCP_server.py 10007
    62 [*] TCP Server listening on port 10007
    63 [+] 192.168.17.34:63824 connected
    64 [+] 192.168.17.34:63824 listed files
    65 [+] 192.168.17.34:63824 downloading ACN.pdf (1394433 bytes)
    66 [+] 192.168.17.34:63824 downloading HTTP_GET.pdf (119428 bytes)
```

Not overwriting an existing file

If the client asks to download a file it already has locally, the client does not overwrite it. Instead it picks a new name automatically (for example appending a number) and saves the download under that name, so nothing already on disk is lost.

```
ftp> get ACN.pdf
⚠️Note: Local file already exists. Auto-renamed destination target to: ACN_2.pdf
✅Success: Downloaded 1394433 bytes to ACN_2.pdf.
🔒Verifying integrity checksum of downloaded file...
SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
ftp> █
```

The screenshot shows a VS Code editor with a file explorer on the left containing a folder named 'socket_prog' with sub-files 'ACN.pdf', 'ACNTut.pdf', 'HTTP_GET.pdf', 'short.pdf', and 'TCP_server.py'. The main editor displays the code for 'TCP_server.py', which is a Python script for a TCP server. The terminal at the bottom shows the command 'python TCP_server.py 10007' and its output, including the server listening on port 10007 and handling a client connection from 192.168.17.34, listing files, and downloading 'ACN.pdf' (1394433 bytes).

```

TCP_server.py
30 def handle_client(conn, addr):
31     try:
32         conn.close()
33         print(f"[-] {client_ip} disconnected")
34     except:
35         pass
36
37 def main():
38     if len(sys.argv) != 2:
39         print("Usage: python3 FTPServer.py <port>")
40         sys.exit(1)
41
42     port = int(sys.argv[1])
43     os.makedirs(STORAGE_DIR, exist_ok=True)
44
45     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
46     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
47     server.bind(('0.0.0.0', port))
48     server.listen(5)
49
50     print(f"[*] TCP Server listening on port {port}")
51     while True:
52         conn, addr = server.accept()
53         thread = threading.Thread(target=handle_client, args=(conn, addr))
54         thread.start()
55
56 if __name__ == '__main__':
57     main()

```

```

PS C:\Users\gajja\Desktop\socket_prog> python TCP_server.py 10007
[*] TCP Server listening on port 10007
[+] 192.168.17.34:63824 connected
[+] 192.168.17.34:63824 listed files
[+] 192.168.17.34:63824 downloading ACN.pdf (1394433 bytes)
[+] 192.168.17.34:63824 downloading HTTP_GET.pdf (119428 bytes)
[+] 192.168.17.34:63824 downloading ACN.pdf (1394433 bytes)

```

Checking that the file arrived correctly

Before uploading, the client computes a SHA-256 hash of the local file. After a download finishes, it hashes the received copy as well and compares the two values. A match confirms the bytes were not corrupted anywhere along the way.

```

PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
✅Connected to the FTP server at 192.168.17.34:10007
ftp> put short.pdf
🔒Calculating local file checksum...
SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
⌚Uploading short.pdf (1394433 bytes)...
✅Success: Transferred 1394433 total bytes cleanly.
ftp> get short.pdf
⚠️Note: Local file already exists. Auto-renamed destination target to: short_1.pdf
✅Success: Downloaded 1394433 bytes to short_1.pdf.
🔒Verifying integrity checksum of downloaded file...
SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
ftp> █

```


Serving more than one client at a time

The server spins up a new thread for every client that connects, so several clients can upload and download at the same time without blocking one another. The two runs below show two separate clients talking to the same server concurrently — one doing a put followed by a get, and the other doing the same with different files.

TCPFTP Client1:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
✓ Connected to the FTP server at 192.168.17.34:10007
ftp> put ACNTut.pdf
🔒 Calculating local file checksum...
  SHA-256: 9c30ce997ef02a565e591c2d67569b34002b186b5e239bf91e94b55463afb48f
⌚ Uploading ACNTut.pdf (423569 bytes)...
✓ Success: Transferred 423569 total bytes cleanly.
ftp> list
📁 Server contains [4] files:
- ACN.pdf 1394433
- ACNTut.pdf 423569
- HTTP_GET.pdf 119428
- short.pdf 1394433
ftp> get ACN.pdf
⚠ Note: Local file already exists. Auto-renamed destination target to: ACN_3.pdf
✓ Success: Downloaded 1394433 bytes to ACN_3.pdf.
🔒 Verifying integrity checksum of downloaded file...
  SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
ftp> 
```

TCPFTP Client2:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python TCPFTPclient.py 192.168.17.34 10007
✓ Connected to the FTP server at 192.168.17.34:10007
ftp> put short.pdf
🔒 Calculating local file checksum...
  SHA-256: e95af85bb0af1392adbf6a3ecb3b87ed6ccd0195a1e830d6b09140bafa8e9ce0
⌚ Uploading short.pdf (1394433 bytes)...
✓ Success: Transferred 1394433 total bytes cleanly.
ftp> list
📁 Server contains [4] files:
- ACN.pdf 1394433
- ACNTut.pdf 423569
- HTTP_GET.pdf 119428
- short.pdf 1394433
ftp> get HTTP_GET.pdf
⚠ Note: Local file already exists. Auto-renamed destination target to: HTTP_GET_2.pdf
✓ Success: Downloaded 119428 bytes to HTTP_GET_2.pdf.
🔒 Verifying integrity checksum of downloaded file...
  SHA-256: 6d3e72dc5244c229c7aa69a946799be4aa709ea75f4f95710464c72a8e5e7fea
ftp> 
```

TCPFTP Server :

The image shows a Visual Studio Code editor window with a Python file named `TCP_server.py` open. The file is located in a workspace named `socket_prog`, under a subfolder `server_storage`. The code implements a simple TCP server that listens on port 10007 and handles client connections using threads.

```
95 def main():
107     print(f"[*] TCP Server listening on port {port}")
108     while True:
109         conn, addr = server.accept()
110         thread = threading.Thread(target=handle_client, args=(conn, addr))
111         thread.start()
112     if __name__ == "__main__":
113         main()
```

The terminal output shows the server running and handling several client connections:

```
PS C:\Users\gajja\Desktop\socket_prog> python TCP_server.py 10007
[*] TCP Server listening on port 10007
[+] 192.168.17.34:63385 connected
[+] 192.168.17.34:63441 connected
[+] 192.168.17.34:63385 uploading ACNTut.pdf (423569 bytes)
[+] Saved ACNTut.pdf
[+] 192.168.17.34:63441 uploading short.pdf (1394433 bytes)
[+] Saved short.pdf
[+] 192.168.17.34:63385 listed files
[+] 192.168.17.34:63441 listed files
[+] 192.168.17.34:63385 downloading ACN.pdf (1394433 bytes)
[+] 192.168.17.34:63441 downloading HTTP_GET.pdf (119428 bytes)
```

On the right side of the editor, there is a chat panel with the heading "Build with Agent". It contains a message: "AI responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase." Below this, there is a section for "TCP_server.py" with a prompt "Describe what to build" and buttons for "Agent", "Models", and "Local".

Part 2 — UDP File Transfer with Reliability

UDP does not guarantee delivery, ordering, or even that a packet arrives at all, so this part builds a small reliability layer on top of it: the file is split into fixed-size chunks, each chunk is numbered, and the receiver acknowledges every chunk it gets. If the sender does not see an acknowledgement in time, it resends that chunk, up to a retry limit. The same put / get / list commands as Part 1 are used, and every completed transfer is verified with an MD5 hash.

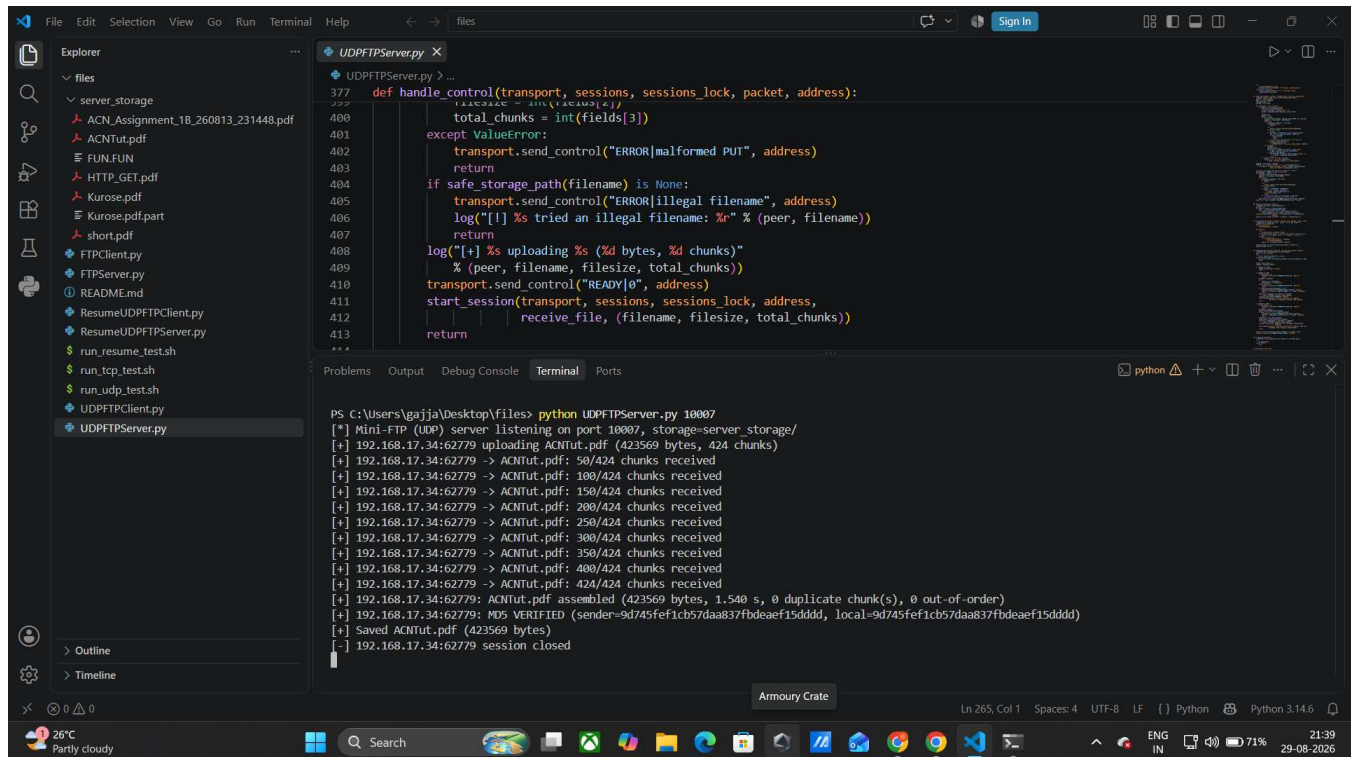
Baseline run with no packet loss

With 0% simulated loss, every chunk is acknowledged on the first try, so there are zero retransmissions and the transfer runs at close to the network's real speed.

UDPFTP Client:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34 10007
Commands: put <file> | get <file> | list | quit
udp-ftp> put ACNTut.pdf
[*] PUT ACNTut.pdf (423569 bytes, 424 chunks), local MD5 = 9d745fef1cb57daa837fbdeaef15dddd
[+] 50/424 chunks acknowledged
[+] 100/424 chunks acknowledged
[+] 150/424 chunks acknowledged
[+] 200/424 chunks acknowledged
[+] 250/424 chunks acknowledged
[+] 300/424 chunks acknowledged
[+] 350/424 chunks acknowledged
[+] 400/424 chunks acknowledged
[+] 424/424 chunks acknowledged
[=] PUT ACNTut.pdf | 423569 bytes | 1.541 s | 0 retransmitted chunk(s) | 268.51 KB/s | VERIFIED
udp-ftp> █
```

UDPFTP Server :



```
def handle_control(transport, sessions, sessions_lock, packet, address):
    fields = parse(fields[3])
    total_chunks = int(fields[3])
    except ValueError:
        transport.send_control("ERROR|malformed PUT", address)
        return
    if safe_storage_path(filename) is None:
        transport.send_control("ERROR|illegal filename", address)
        log("[!] %s tried an illegal filename: %r" % (peer, filename))
        return
    log("[+] %s uploading %s (%d bytes, %d chunks)"
        % (peer, filename, filesize, total_chunks))
    transport.send_control("READY|0", address)
    start_session(transport, sessions, sessions_lock, address,
        receive_file, (filename, filesize, total_chunks))
    return

PS C:\Users\gajja\Desktop\files> python UDPFTPServer.py 10007
[*] Mini-FTP (UDP) server listening on port 10007, storage=server_storage/
[+] 192.168.17.34:62779 uploading ACNTut.pdf (423569 bytes, 424 chunks)
[+] 192.168.17.34:62779 -> ACNTut.pdf: 50/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf: 100/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf: 150/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf: 200/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf: 250/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf: 300/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf: 350/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf: 400/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf: 424/424 chunks received
[+] 192.168.17.34:62779 -> ACNTut.pdf assembled (423569 bytes, 1.540 s, 0 duplicate chunk(s), 0 out-of-order)
[+] 192.168.17.34:62779: ACNTut.pdf assembled (423569 bytes, 1.540 s, 0 duplicate chunk(s), 0 out-of-order)
[+] 192.168.17.34:62779: MD5 VERIFIED (sender=9d745fef1cb57daa837fbdeaef15dddd, local=9d745fef1cb57daa837fbdeaef15dddd)
[+] Saved ACNTut.pdf (423569 bytes)
[+] 192.168.17.34:62779 session closed
```

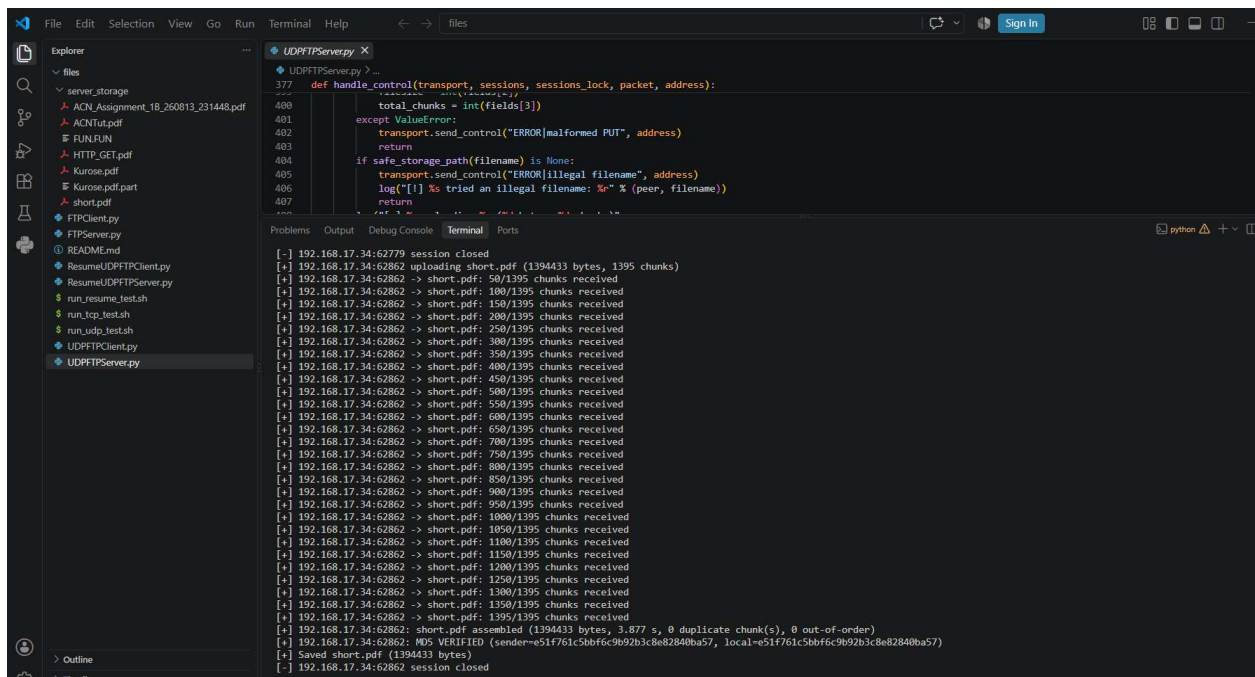
A larger file (over 1 MB)

The same put command was repeated with a bigger file to check that chunking and reassembly still work correctly once there are well over a thousand chunks to track.

UDPFTP Client:

```
PS C:\Users\sai\h\Desktop\Socket Programming-2> python UDPFTPClient.py 192.168.17.34 10007
Commands: put <file> | get <file> | list | quit
udp-ftp> put short.pdf
[*] PUT short.pdf (1394433 bytes, 1395 chunks), local MD5 = e51f761c5bbf6c9b92b3c8e82840ba57
[+] 50/1395 chunks acknowledged
[+] 100/1395 chunks acknowledged
[+] 150/1395 chunks acknowledged
[+] 200/1395 chunks acknowledged
[+] 250/1395 chunks acknowledged
[+] 300/1395 chunks acknowledged
[+] 350/1395 chunks acknowledged
[+] 400/1395 chunks acknowledged
[+] 450/1395 chunks acknowledged
[+] 500/1395 chunks acknowledged
[+] 550/1395 chunks acknowledged
[+] 600/1395 chunks acknowledged
[+] 650/1395 chunks acknowledged
[+] 700/1395 chunks acknowledged
[+] 750/1395 chunks acknowledged
[+] 800/1395 chunks acknowledged
[+] 850/1395 chunks acknowledged
[+] 900/1395 chunks acknowledged
[+] 950/1395 chunks acknowledged
[+] 1000/1395 chunks acknowledged
[+] 1050/1395 chunks acknowledged
[+] 1100/1395 chunks acknowledged
[+] 1150/1395 chunks acknowledged
[+] 1200/1395 chunks acknowledged
[+] 1250/1395 chunks acknowledged
[+] 1300/1395 chunks acknowledged
[+] 1350/1395 chunks acknowledged
[+] 1395/1395 chunks acknowledged
[=] PUT short.pdf | 1394433 bytes | 3.881 s | 0 retransmitted chunk(s) | 350.86 KB/s | VERIFIED
udp-ftp> |
```

UDPFTP Server :



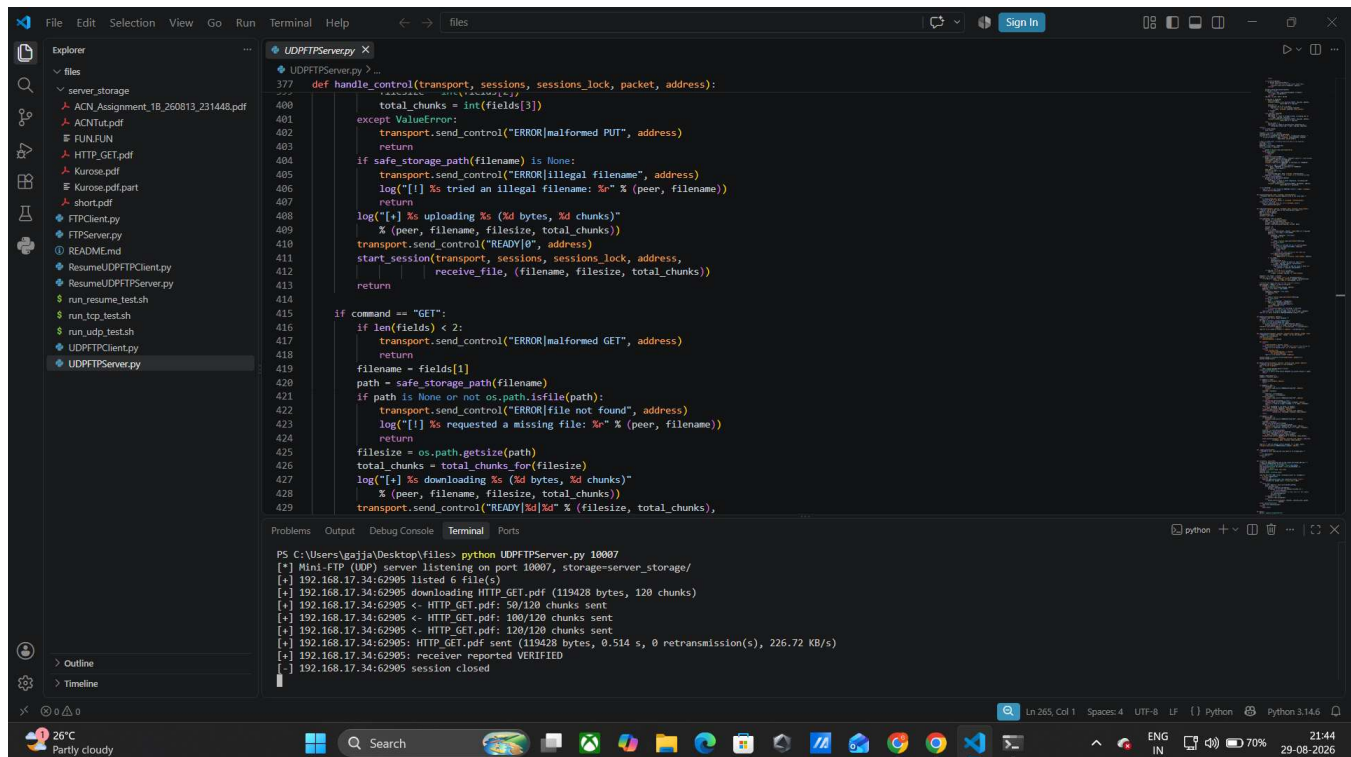
Downloading files

The get command works the same way in reverse: the server chunks the file it is sending, and the client acknowledges each chunk as it arrives, both for a small file and for one over 1 MB.

UDPFFTP Client:

```
PS C:\Users\sai ch\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34 10007
Commands: put <file> | get <file> | list | quit
udp-ftp> list
[*] 6 file(s) on the server:
    ACNTut.pdf                423569 bytes
    ACN_Assignment_1B_260813_231448.pdf 1394433 bytes
    FUN.FUN                   45 bytes
    HTTP_GET.pdf              119428 bytes
    Kurose.pdf                19661364 bytes
    short.pdf                 1394433 bytes
udp-ftp> get HTTP_GET.pdf
[*] HTTP_GET.pdf already exists locally; saving as HTTP_GET (1).pdf
[*] GET HTTP_GET.pdf (119428 bytes, 120 chunks) -> HTTP_GET (1).pdf
[+] 50/120 chunks received
[+] 100/120 chunks received
[+] 120/120 chunks received
[*] 0 duplicate chunk(s), 0 out-of-order chunk(s)
[=] GET HTTP_GET.pdf | 119428 bytes | 0.501 s | 0 retransmitted chunk(s) | 232.83 KB/s | VERIFIED
[+] Local MD5 = f89433fcb726efe91d2e41c734465cf
udp-ftp> |
```

UDPFFTP Server :



```
def handle_control(transport, sessions, sessions_lock, packet, address):
    ...
    total_chunks = int(fields[3])
    except ValueError:
        transport.send_control("ERROR|malformed PUT", address)
        return
    if safe_storage_path(filename) is None:
        transport.send_control("ERROR|illegal filename", address)
        log("[!] %s tried an illegal filename: %s" % (peer, filename))
        return
    log("[+] %s uploading %s (%d bytes, %d chunks)"
        % (peer, filename, filesize, total_chunks))
    transport.send_control("READY|0", address)
    start_session(transport, sessions, sessions_lock, address,
        receive_file, (filename, filesize, total_chunks))
    return

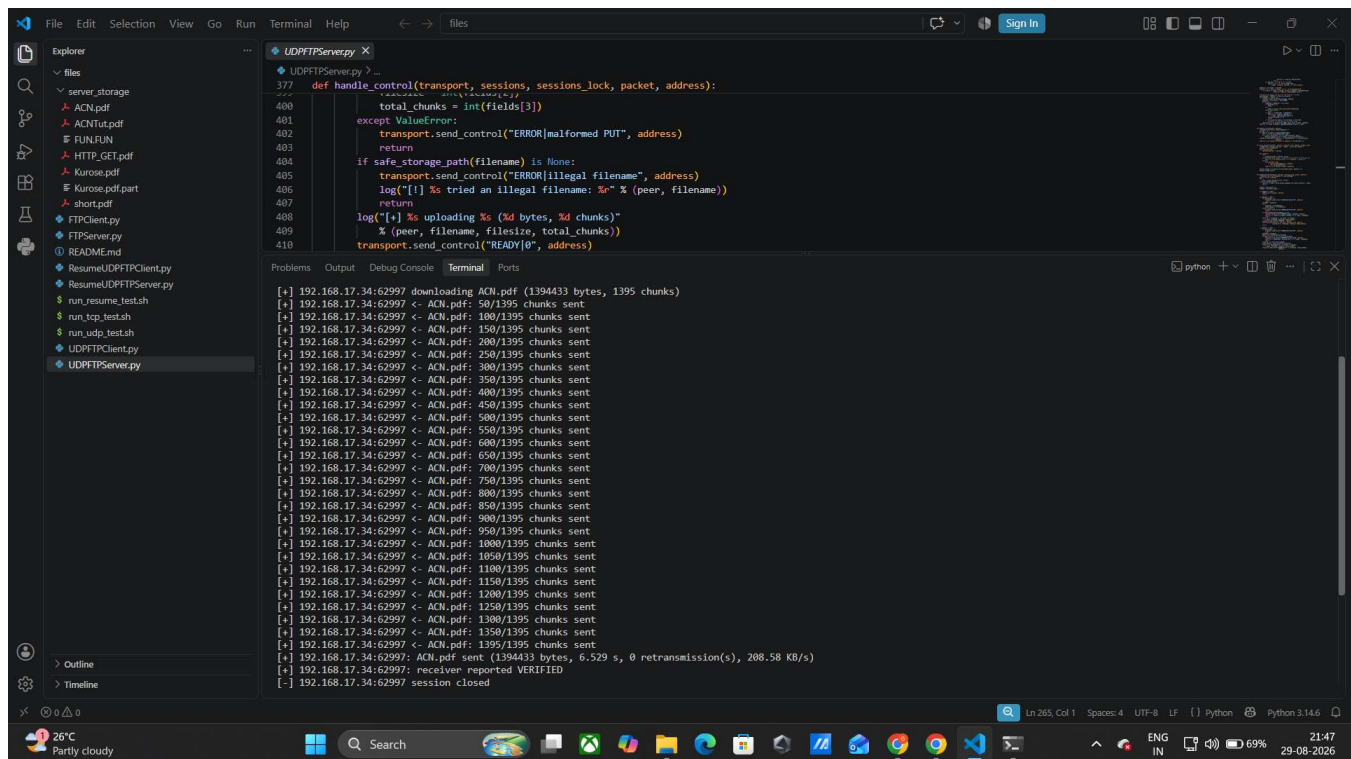
if command == "GET":
    if len(fields) < 2:
        transport.send_control("ERROR|malformed GET", address)
        return
    filename = fields[1]
    path = safe_storage_path(filename)
    if path is None or not os.path.isfile(path):
        transport.send_control("ERROR|file not found", address)
        log("[!] %s requested a missing file: %s" % (peer, filename))
        return
    filesize = os.path.getsize(path)
    total_chunks = total_chunks_for(filesize)
    log("[+] %s downloading %s (%d bytes, %d chunks)"
        % (peer, filename, filesize, total_chunks))
    transport.send_control("READY|%d" % (filesize, total_chunks),
```

```
PS C:\Users\gajja\Desktop\files> python UDPFTPServer.py 10007
[*] Mini-FTP (UDP) server listening on port 10007, storage=server_storage/
[+] 192.168.17.34:62905 listed 6 file(s)
[+] 192.168.17.34:62905 downloading HTTP_GET.pdf (119428 bytes, 120 chunks)
[+] 192.168.17.34:62905 <- HTTP_GET.pdf: 50/120 chunks sent
[+] 192.168.17.34:62905 <- HTTP_GET.pdf: 100/120 chunks sent
[+] 192.168.17.34:62905 <- HTTP_GET.pdf: 120/120 chunks sent
[+] 192.168.17.34:62905: HTTP_GET.pdf sent (119428 bytes, 0.514 s, 0 retransmission(s), 226.72 KB/s)
[+] 192.168.17.34:62905: receiver reported VERIFIED
[-] 192.168.17.34:62905 session closed
```

UDPFTP client :

```
PS C:\Users\sach\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34 10007
Commands: put <file> | get <file> | list | quit
udp-ftp> get ACN.pdf
[*] ACN.pdf already exists locally; saving as ACN (1).pdf
[*] GET ACN.pdf (1394433 bytes, 1395 chunks) -> ACN (1).pdf
[+] 50/1395 chunks received
[+] 100/1395 chunks received
[+] 150/1395 chunks received
[+] 200/1395 chunks received
[+] 250/1395 chunks received
[+] 300/1395 chunks received
[+] 350/1395 chunks received
[+] 400/1395 chunks received
[+] 450/1395 chunks received
[+] 500/1395 chunks received
[+] 550/1395 chunks received
[+] 600/1395 chunks received
[+] 650/1395 chunks received
[+] 700/1395 chunks received
[+] 750/1395 chunks received
[+] 800/1395 chunks received
[+] 850/1395 chunks received
[+] 900/1395 chunks received
[+] 950/1395 chunks received
[+] 1000/1395 chunks received
[+] 1050/1395 chunks received
[+] 1100/1395 chunks received
[+] 1150/1395 chunks received
[+] 1200/1395 chunks received
[+] 1250/1395 chunks received
[+] 1300/1395 chunks received
[+] 1350/1395 chunks received
[+] 1395/1395 chunks received
[*] 0 duplicate chunk(s), 0 out-of-order chunk(s)
[=] GET ACN.pdf | 1394433 bytes | 6.530 s | 0 retransmitted chunk(s) | 208.54 KB/s | VERIFIED
[+] Local MD5 = e51f761c5bbf6c9b92b3c8e82840ba57
udp-ftp> █
```

UDPFTP Server:



The screenshot displays a Windows IDE with the file explorer on the left showing a project structure with files like ACN.pdf, ACNTut.pdf, and various test scripts. The main editor shows the UDPFTPServer.py code, which includes a `def handle_control` function for managing file transfers. The terminal at the bottom shows the server's output, including the download of ACN.pdf (1394433 bytes, 1395 chunks) and the receipt of 1395 chunks from the client. The status bar at the bottom indicates the system is at 26°C, 69% battery, and the date is 29-08-2026.

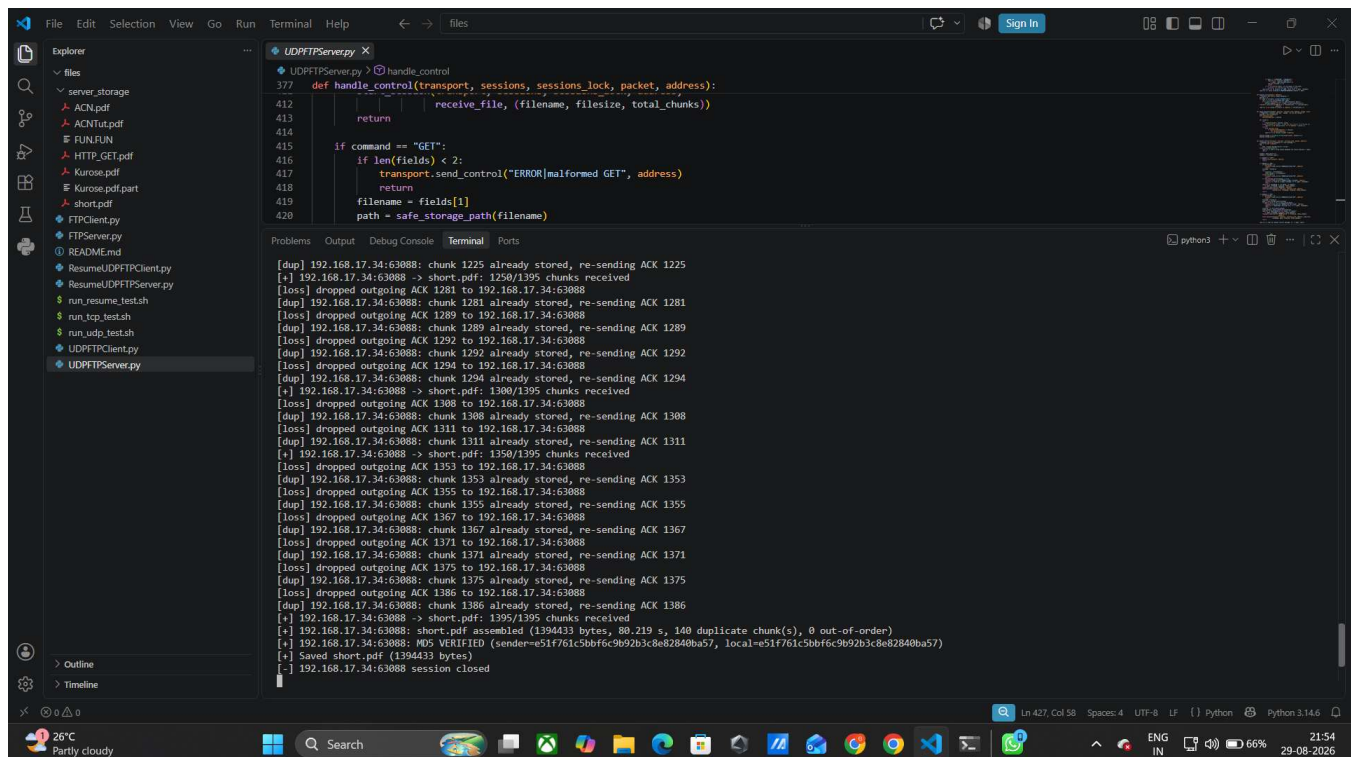
```
File Edit Selection View Go Run Terminal Help
UDPFTPServer.py
def handle_control(transport, sessions, sessions_lock, packet, address):
    377     total_chunks = int(fields[3])
    378 except ValueError:
    379     transport.send_control("ERROR|malformed PUT", address)
    380     return
    381 if safe_storage_path(filename) is None:
    382     transport.send_control("ERROR|illegal filename", address)
    383     log("[!] %s tried an illegal filename: %s" % (peer, filename))
    384     return
    385 log("[+] %s uploading %s (%d bytes, %d chunks)"
    386     % (peer, filename, filesize, total_chunks))
    387 transport.send_control("READY|0", address)
    388
    389
    390
    391
    392
    393
    394
    395
    396
    397
    398
    399
    400
    401
    402
    403
    404
    405
    406
    407
    408
    409
    410
    411
    412
    413
    414
    415
    416
    417
    418
    419
    420
    421
    422
    423
    424
    425
    426
    427
    428
    429
    430
    431
    432
    433
    434
    435
    436
    437
    438
    439
    440
    441
    442
    443
    444
    445
    446
    447
    448
    449
    450
    451
    452
    453
    454
    455
    456
    457
    458
    459
    460
    461
    462
    463
    464
    465
    466
    467
    468
    469
    470
    471
    472
    473
    474
    475
    476
    477
    478
    479
    480
    481
    482
    483
    484
    485
    486
    487
    488
    489
    490
    491
    492
    493
    494
    495
    496
    497
    498
    499
    500
    501
    502
    503
    504
    505
    506
    507
    508
    509
    510
    511
    512
    513
    514
    515
    516
    517
    518
    519
    520
    521
    522
    523
    524
    525
    526
    527
    528
    529
    530
    531
    532
    533
    534
    535
    536
    537
    538
    539
    540
    541
    542
    543
    544
    545
    546
    547
    548
    549
    550
    551
    552
    553
    554
    555
    556
    557
    558
    559
    560
    561
    562
    563
    564
    565
    566
    567
    568
    569
    570
    571
    572
    573
    574
    575
    576
    577
    578
    579
    580
    581
    582
    583
    584
    585
    586
    587
    588
    589
    590
    591
    592
    593
    594
    595
    596
    597
    598
    599
    600
    601
    602
    603
    604
    605
    606
    607
    608
    609
    610
    611
    612
    613
    614
    615
    616
    617
    618
    619
    620
    621
    622
    623
    624
    625
    626
    627
    628
    629
    630
    631
    632
    633
    634
    635
    636
    637
    638
    639
    640
    641
    642
    643
    644
    645
    646
    647
    648
    649
    650
    651
    652
    653
    654
    655
    656
    657
    658
    659
    660
    661
    662
    663
    664
    665
    666
    667
    668
    669
    670
    671
    672
    673
    674
    675
    676
    677
    678
    679
    680
    681
    682
    683
    684
    685
    686
    687
    688
    689
    690
    691
    692
    693
    694
    695
    696
    697
    698
    699
    700
    701
    702
    703
    704
    705
    706
    707
    708
    709
    710
    711
    712
    713
    714
    715
    716
    717
    718
    719
    720
    721
    722
    723
    724
    725
    726
    727
    728
    729
    730
    731
    732
    733
    734
    735
    736
    737
    738
    739
    740
    741
    742
    743
    744
    745
    746
    747
    748
    749
    750
    751
    752
    753
    754
    755
    756
    757
    758
    759
    760
    761
    762
    763
    764
    765
    766
    767
    768
    769
    770
    771
    772
    773
    774
    775
    776
    777
    778
    779
    780
    781
    782
    783
    784
    785
    786
    787
    788
    789
    790
    791
    792
    793
    794
    795
    796
    797
    798
    799
    800
    801
    802
    803
    804
    805
    806
    807
    808
    809
    810
    811
    812
    813
    814
    815
    816
    817
    818
    819
    820
    821
    822
    823
    824
    825
    826
    827
    828
    829
    830
    831
    832
    833
    834
    835
    836
    837
    838
    839
    840
    841
    842
    843
    844
    845
    846
    847
    848
    849
    850
    851
    852
    853
    854
    855
    856
    857
    858
    859
    860
    861
    862
    863
    864
    865
    866
    867
    868
    869
    870
    871
    872
    873
    874
    875
    876
    877
    878
    879
    880
    881
    882
    883
    884
    885
    886
    887
    888
    889
    890
    891
    892
    893
    894
    895
    896
    897
    898
    899
    900
    901
    902
    903
    904
    905
    906
    907
    908
    909
    910
    911
    912
    913
    914
    915
    916
    917
    918
    919
    920
    921
    922
    923
    924
    925
    926
    927
    928
    929
    930
    931
    932
    933
    934
    935
    936
    937
    938
    939
    940
    941
    942
    943
    944
    945
    946
    947
    948
    949
    950
    951
    952
    953
    954
    955
    956
    957
    958
    959
    960
    961
    962
    963
    964
    965
    966
    967
    968
    969
    970
    971
    972
    973
    974
    975
    976
    977
    978
    979
    980
    981
    982
    983
    984
    985
    986
    987
    988
    989
    990
    991
    992
    993
    994
    995
    996
    997
    998
    999
    1000
    1001
    1002
    1003
    1004
    1005
    1006
    1007
    1008
    1009
    1010
    1011
    1012
    1013
    1014
    1015
    1016
    1017
    1018
    1019
    1020
    1021
    1022
    1023
    1024
    1025
    1026
    1027
    1028
    1029
    1030
    1031
    1032
    1033
    1034
    1035
    1036
    1037
    1038
    1039
    1040
    1041
    1042
    1043
    1044
    1045
    1046
    1047
    1048
    1049
    1050
    1051
    1052
    1053
    1054
    1055
    1056
    1057
    1058
    1059
    1060
    1061
    1062
    1063
    1064
    1065
    1066
    1067
    1068
    1069
    1070
    1071
    1072
    1073
    1074
    1075
    1076
    1077
    1078
    1079
    1080
    1081
    1082
    1083
    1084
    1085
    1086
    1087
    1088
    1089
    1090
    1091
    1092
    1093
    1094
    1095
    1096
    1097
    1098
    1099
    1100
    1101
    1102
    1103
    1104
    1105
    1106
    1107
    1108
    1109
    1110
    1111
    1112
    1113
    1114
    1115
    1116
    1117
    1118
    1119
    1120
    1121
    1122
    1123
    1124
    1125
    1126
    1127
    1128
    1129
    1130
    1131
    1132
    1133
    1134
    1135
    1136
    1137
    1138
    1139
    1140
    1141
    1142
    1143
    1144
    1145
    1146
    1147
    1148
    1149
    1150
    1151
    1152
    1153
    1154
    1155
    1156
    1157
    1158
    1159
    1160
    1161
    1162
    1163
    1164
    1165
    1166
    1167
    1168
    1169
    1170
    1171
    1172
    1173
    1174
    1175
    1176
    1177
    1178
    1179
    1180
    1181
    1182
    1183
    1184
    1185
    1186
    1187
    1188
    1189
    1190
    1191
    1192
    1193
    1194
    1195
    1196
    1197
    1198
    1199
    1200
    1201
    1202
    1203
    1204
    1205
    1206
    1207
    1208
    1209
    1210
    1211
    1212
    1213
    1214
    1215
    1216
    1217
    1218
    1219
    1220
    1221
    1222
    1223
    1224
    1225
    1226
    1227
    1228
    1229
    1230
    1231
    1232
    1233
    1234
    1235
    1236
    1237
    1238
    1239
    1240
    1241
    1242
    1243
    1244
    1245
    1246
    1247
    1248
    1249
    1250
    1251
    1252
    1253
    1254
    1255
    1256
    1257
    1258
    1259
    1260
    1261
    1262
    1263
    1264
    1265
    1266
    1267
    1268
    1269
    1270
    1271
    1272
    1273
    1274
    1275
    1276
    1277
    1278
    1279
    1280
    1281
    1282
    1283
    1284
    1285
    1286
    1287
    1288
    1289
    1290
    1291
    1292
    1293
    1294
    1295
    1296
    1297
    1298
    1299
    1300
    1301
    1302
    1303
    1304
    1305
    1306
    1307
    1308
    1309
    1310
    1311
    1312
    1313
    1314
    1315
    1316
    1317
    1318
    1319
    1320
    1321
    1322
    1323
    1324
    1325
    1326
    1327
    1328
    1329
    1330
    1331
    1332
    1333
    1334
    1335
    1336
    1337
    1338
    1339
    1340
    1341
    1342
    1343
    1344
    1345
    1346
    1347
    1348
    1349
    1350
    1351
    1352
    1353
    1354
    1355
    1356
    1357
    1358
    1359
    1360
    1361
    1362
    1363
    1364
    1365
    1366
    1367
    1368
    1369
    1370
    1371
    1372
    1373
    1374
    1375
    1376
    1377
    1378
    1379
    1380
    1381
    1382
    1383
    1384
    1385
    1386
    1387
    1388
    1389
    1390
    1391
    1392
    1393
    1394
    1395
    1396
    1397
    1398
    1399
    1400
    1401
    1402
    1403
    1404
    1405
    1406
    1407
    1408
    1409
    1410
    1411
    1412
    1413
    1414
    1415
    1416
    1417
    1418
    1419
    1420
    1421
    1422
    1423
    1424
    1425
    1426
    1427
    1428
    1429
    1430
    1431
    1432
    1433
    1434
    1435
    1436
    1437
    1438
    1439
    1440
    1441
    1442
    1443
    1444
    1445
    1446
    1447
    1448
    1449
    1450
    1451
    1452
    1453
    1454
    1455
    1456
    1457
    1458
    1459
    1460
    1461
    1462
    1463
    1464
    1465
    1466
    1467
    1468
    1469
    1470
    1471
    1472
    1473
    1474
    1475
    1476
    1477
    1478
    1479
    1480
    1481
    1482
    1483
    1484
    1485
    1486
    1487
    1488
    1489
    1490
    1491
    1492
    1493
    1494
    1495
    1496
    1497
    1498
    1499
    1500
    1501
    1502
    1503
    1504
    1505
    1506
    1507
    1508
    1509
    1510
    1511
    1512
    1513
    1514
    1515
    1516
    1517
    1518
    1519
    1520
    1521
    1522
    1523
    1524
    1525
    1526
    1527
    1528
    1529
    1530
    1531
    1532
    1533
    1534
    1535
    1536
    1537
    1538
    1539
    1540
    1541
    1542
    1543
    1544
    1545
    1546
    1547
    1548
    1549
    1550
    1551
    1552
    1553
    1554
    1555
    1556
    1557
    1558
    1559
    1560
    1561
    1562
    1563
    1564
    1565
    1566
    1567
    1568
    1569
    1570
    1571
    1572
    1573
    1574
    1575
    1576
    1577
    1578
    1579
    1580
    1581
    1582
    1583
    1584
    1585
    1586
    1587
    1588
    1589
    1590
    1591
    1592
    1593
    1594
    1595
    1596
    1597
    1598
    1599
    1600
    1601
    1602
    1603
    1604
    1605
    1606
    1607
    1608
    1609
    1610
    1611
    1612
    1613
    1614
    1615
    1616
    1617
    1618
    1619
    1620
    1621
    1622
    1623
    1624
    1625
    1626
    1627
    1628
    1629
    1630
    1631
    1632
    1633
    1634
    1635
    1636
    1637
    1638
    1639
    1640
    1641
    1642
    1643
    1644
    1645
    1646
    1647
    1648
    1649
    1650
    1651
    1652
    1653
    1654
    1655
    1656
    1657
    1658
    1659
    1660
    1661
    1662
    1663
    1664
    1665
    1666
    1667
    1668
    1669
    1670
    1671
    1672
    1673
    1674
    1675
    1676
    1677
    1678
    1679
    1680
    1681
    1682
    1683
    1684
    1685
    1686
    1687
    1688
    1689
    1690
    1691
    1692
    1693
    1694
    1695
    1696
    1697
    1698
    1699
    1700
    1701
    1702
    1703
    1704
    1705
    1706
    1707
    1708
    1709
    1710
    1711
    1712
    1713
    1714
    1715
    1716
    1717
    1718
    1719
    1720
    1721
    1722
    1723
    1724
    1725
    1726
    1727
    1728
    1729
    1730
    1731
    1732
    1733
    1734
    1735
    1736
    1737
    1738
    1739
    1740
    1741
    1742
    1743
    1744
    1745
    1746
    1747
    1748
    1749
    1750
    1751
    1752
    1753
    1754
    1755
    1756
    1757
    1758
    1759
    1760
    1761
    1762
    1763
    1764
    1765
    1766
    1767
    1768
    1769
    1770
    1771
    1772
    1773
    1774
    1775
    1776
    1777
    1778
    1779
    1780
    1781
    1782
    1783
    1784
    1785
    1786
    1787
    1788
    1789
    1790
    1791
    1792
    1793
    1794
    1795
    1796
    1797
    1798
    1799
    1800
    1801
    1802
    1803
    1804
    1805
    1806
    1807
    1808
    1809
    1810
    1811
    1812
    1813
    1814
    1815
    1816
    1817
    1818
    1819
    1820
    1821
    1822
    1823
    1824
    1825
    1826
    1827
    1828
    1829
    1830
    1831
    1832
    1833
    1834
    1835
    1836
    1837
    1838
    1839
    1840
    1841
    1842
    1843
    1844
    1845
    1846
    1847
    1848
    1849
    1850
    1851
    1852
    1853
    1854
    1855
    1856
    1857
    1858
    1859
    1860
    1861
    1862
    1863
    1864
    1865
    1866
    1867
    1868
    1869
    1870
    1871
    1872
    1873
    1874
    1875
    1876
    1877
    1878
    1879
    1880
    1881
    1882
    1883
    1884
    1885
    1886
    1887
    1888
    1889
    1890
    1891
    1892
    1893
    1894
    1895
    1896
    1897
    1898
    1899
    1900
    1901
    1902
    1903
    1904
    1905
    1906
    1907
    1908
    1909
    1910
    1911
    1912
    1913
    1914
    1915
    1916
    1917
    1918
    1919
    1920
    1921
    1922
    1923
    1924
    1925
    1926
    1927
    1928
    1929
    1930
    1931
    1932
    1933
    1934
    1935
    1936
    1937
    1938
    1939
    1940
    1941
    1942
    1943
    1944
    1945
    1946
    1947
    1948
    1949
    1950
    1951
    1952
    1953
    1954
    1955
    1956
    1957
    1958
    1959
    1960
    1961
    1962
    1963
    1964
    1965
    1966
    1967
    1968
    1969
    1970
    1971
    1972
    1973
    1974
    1975
    1976
    1977
    1978
    1979
    1980
    1981
    1982
    1983
    1984
    1985
    1986
    1987
    1988
    1989
    1990
    1991
    1992
    1993
    1994
    1995
    1996
    1997
    1998
    1999
    2000
    2001
    2002
    2003
    2004
    2005
    2006
    2007
    2008
    2009
    2010
    2011
    2012
    2013
    2014
    2015
    2016
    2017
    2018
    2019
    2020
    2021
    2022
    2023
    2024
    2025
    2026
    2027
    2028
    2029
    2030
    2031
    2032
    2033
    2034
    2035
    2036
    2037
    2038
    2039
    2040
    2041
    2042
    2043
    2044
    2045
    2046
    2047
    2048
    2049
    2050
    2051
    2052
    2053
    2054
    2055
    2056
    2057
    2058
    2059
    2060
    2061
    2062
    2063
    2064
    2065
    2066
    2067
    2068
    2069
    2070
    2071
    2072
    2073
    2074
    2075
    2076
    2077
```

To test the retry logic, packet loss was simulated at two levels: 10% and 20%. At these rates, some chunks (or their acknowledgements) are deliberately dropped, forcing the sender to time out and resend. Each affected chunk is retried up to 10 times before giving up.

10% loss — upload (PUT) (UDPFFTP Client)

```
PS C:\Users\saich\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34 10007
[+] 1150/1395 chunks acknowledged
[rtx] timeout on chunk 1153, retry 1/10
[rtx] timeout on chunk 1161, retry 1/10
[rtx] timeout on chunk 1167, retry 1/10
[rtx] timeout on chunk 1171, retry 1/10
[rtx] timeout on chunk 1173, retry 1/10
[rtx] timeout on chunk 1173, retry 2/10
[rtx] timeout on chunk 1175, retry 1/10
[rtx] timeout on chunk 1194, retry 1/10
[+] 1200/1395 chunks acknowledged
[rtx] timeout on chunk 1225, retry 1/10
[+] 1250/1395 chunks acknowledged
[rtx] timeout on chunk 1281, retry 1/10
[rtx] timeout on chunk 1289, retry 1/10
[rtx] timeout on chunk 1292, retry 1/10
[rtx] timeout on chunk 1294, retry 1/10
[+] 1300/1395 chunks acknowledged
[rtx] timeout on chunk 1308, retry 1/10
[rtx] timeout on chunk 1311, retry 1/10
[+] 1350/1395 chunks acknowledged
[rtx] timeout on chunk 1353, retry 1/10
[rtx] timeout on chunk 1355, retry 1/10
[rtx] timeout on chunk 1367, retry 1/10
[rtx] timeout on chunk 1371, retry 1/10
[rtx] timeout on chunk 1375, retry 1/10
[rtx] timeout on chunk 1386, retry 1/10
[+] 1395/1395 chunks acknowledged
[=] PUT short.pdf | 1394433 bytes | 80.224 s | 140 retransmitted chunk(s) | 16.97 KB/s | VERIFIED
udp-ftp> █
```

UDPFFTP Server:



```
def handle_control(transport, sessions, sessions_lock, packet, address):
    receive_file(filename, filesize, total_chunks))
    return
    if command == "GET":
        if len(fields) < 2:
            transport.send_control("ERROR[malformed GET", address)
            return
        filename = fields[1]
        path = safe_storage_path(filename)
    # ... (rest of the code) ...

[+] 192.168.17.34:63088: chunk 1225 already stored, re-sending ACK 1225
[+] 192.168.17.34:63088 -> short.pdf: 1250/1395 chunks received
[loss] dropped outgoing ACK 1281 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1281 already stored, re-sending ACK 1281
[loss] dropped outgoing ACK 1289 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1289 already stored, re-sending ACK 1289
[loss] dropped outgoing ACK 1292 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1292 already stored, re-sending ACK 1292
[loss] dropped outgoing ACK 1294 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1294 already stored, re-sending ACK 1294
[+] 192.168.17.34:63088 -> short.pdf: 1300/1395 chunks received
[loss] dropped outgoing ACK 1308 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1308 already stored, re-sending ACK 1308
[loss] dropped outgoing ACK 1311 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1311 already stored, re-sending ACK 1311
[+] 192.168.17.34:63088 -> short.pdf: 1350/1395 chunks received
[loss] dropped outgoing ACK 1353 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1353 already stored, re-sending ACK 1353
[loss] dropped outgoing ACK 1355 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1355 already stored, re-sending ACK 1355
[loss] dropped outgoing ACK 1367 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1367 already stored, re-sending ACK 1367
[loss] dropped outgoing ACK 1371 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1371 already stored, re-sending ACK 1371
[loss] dropped outgoing ACK 1375 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1375 already stored, re-sending ACK 1375
[loss] dropped outgoing ACK 1386 to 192.168.17.34:63088
[+] 192.168.17.34:63088: chunk 1386 already stored, re-sending ACK 1386
[+] 192.168.17.34:63088 -> short.pdf: 1395/1395 chunks received
[+] 192.168.17.34:63088: short.pdf assembled (1394433 bytes, 80.219 s, 140 duplicate chunk(s), 0 out-of-order)
[+] 192.168.17.34:63088: MD5 VERIFIED (sender=e51f761c5bbf6c9b92b3c8e82840ba57, local=e51f761c5bbf6c9b92b3c8e82840ba57)
[+] Saved short.pdf (1394433 bytes)
[+] 192.168.17.34:63088 session closed
```


10% loss — download (GET) (UDPFTP Client)

```
PS C:\Users\saihc\Desktop\Socket Programming-2> python UDPFTPclient.py 192.168.17.34 10007
[+] 100/1395 chunks received
[+] 200/1395 chunks received
[+] 250/1395 chunks received
[+] 300/1395 chunks received
[+] 350/1395 chunks received
[+] 400/1395 chunks received
[+] 450/1395 chunks received
[+] 500/1395 chunks received
[+] 550/1395 chunks received
[+] 600/1395 chunks received
[+] 650/1395 chunks received
[+] 700/1395 chunks received
[+] 750/1395 chunks received
[+] 800/1395 chunks received
[+] 850/1395 chunks received
[+] 900/1395 chunks received
[+] 950/1395 chunks received
[+] 1000/1395 chunks received
[+] 1050/1395 chunks received
[+] 1100/1395 chunks received
[+] 1150/1395 chunks received
[+] 1200/1395 chunks received
[+] 1250/1395 chunks received
[+] 1300/1395 chunks received
[+] 1350/1395 chunks received
[+] 1395/1395 chunks received
[*] 0 duplicate chunk(s), 0 out-of-order chunk(s)
[=] GET ACN.pdf | 1394433 bytes | 90.350 s | 0 retransmitted chunk(s) | 15.07 KB/s | VERIFIED
[+] Local MD5 = e51f761c5bbf6c9b92b3c8e82840ba57
udp-ftp> |
```

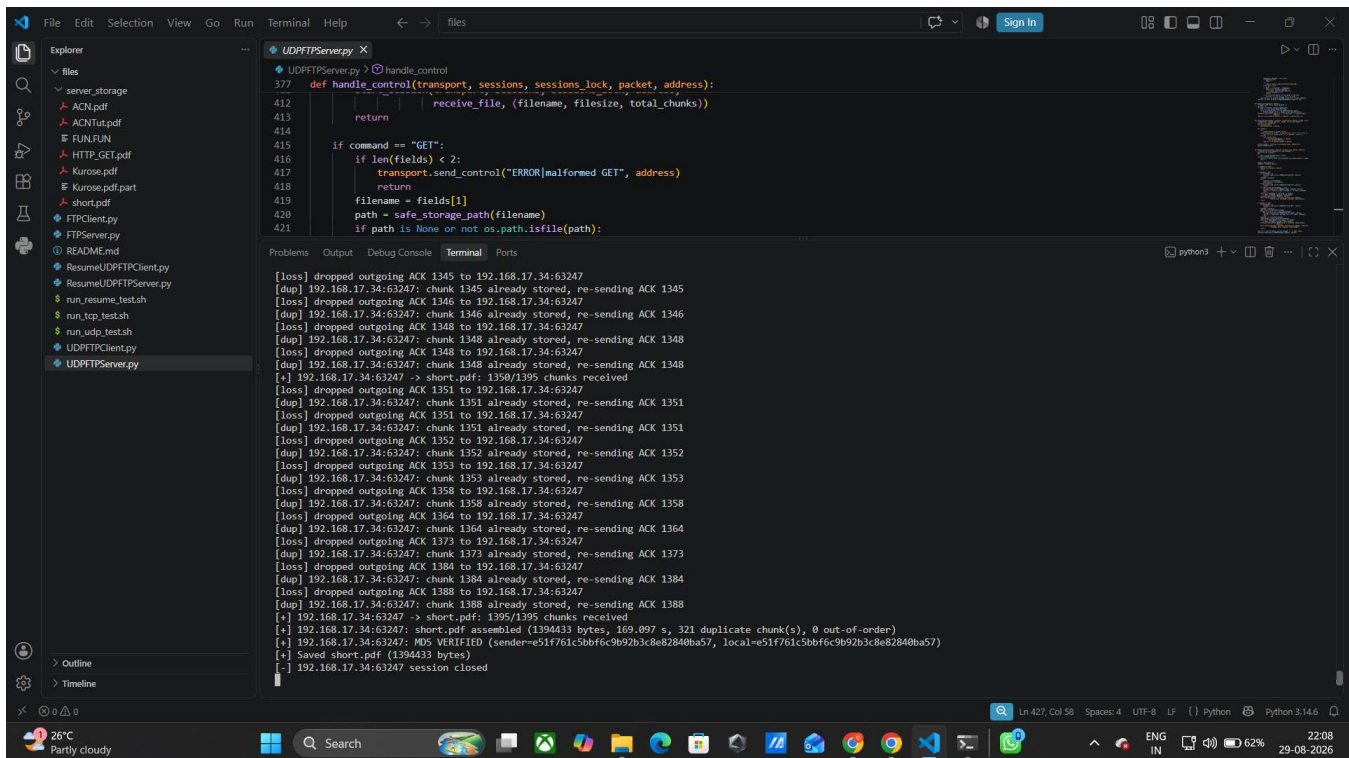
UDPFTP Server:

The screenshot shows the Visual Studio Code interface with the `UDPFTPServer.py` file open. The Explorer pane on the left lists the project files. The main editor displays the `handle_control` function, which manages file transfers. The terminal at the bottom shows the server's log, indicating successful downloads of `ACN.pdf` with a 10% loss rate and a total size of 1,394,433 bytes. The log also shows various timeouts and retries for different data chunks.

20% loss — upload (PUT) (UDPFTP Client)

```
[rtx] timeout on chunk 1348, retry 2/10
[+] 1350/1395 chunks acknowledged
[rtx] timeout on chunk 1351, retry 1/10
[rtx] timeout on chunk 1351, retry 2/10
[rtx] timeout on chunk 1352, retry 1/10
[rtx] timeout on chunk 1353, retry 1/10
[rtx] timeout on chunk 1358, retry 1/10
[rtx] timeout on chunk 1364, retry 1/10
[rtx] timeout on chunk 1373, retry 1/10
[rtx] timeout on chunk 1384, retry 1/10
[rtx] timeout on chunk 1388, retry 1/10
[+] 1395/1395 chunks acknowledged
[=] PUT short.pdf | 1394433 bytes | 169.101 s | 321 retransmitted chunk(s) | 8.05 KB/s | VERIFIED
udp-ftp> █
```

UDPFTP Server:



```
def handle_control(self, transport, sessions, sessions_lock, packet, address):
    def receive_file(filename, filesize, total_chunks):
        # ... (code omitted) ...
    return

    if command == "GET":
        if len(fields) < 2:
            transport.send_control("ERROR[malformed GET]", address)
            return
        filename = fields[1]
        path = safe_storage_path(filename)
        if path is None or not os.path.isfile(path):
```

```
[loss] dropped outgoing ACK 1345 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1345 already stored, re-sending ACK 1345
[loss] dropped outgoing ACK 1346 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1346 already stored, re-sending ACK 1346
[loss] dropped outgoing ACK 1348 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1348 already stored, re-sending ACK 1348
[loss] dropped outgoing ACK 1348 to 192.168.17.34:63247
[+] 192.168.17.34:63247 -> short.pdf: 1350/1395 chunks received
[loss] dropped outgoing ACK 1351 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1351 already stored, re-sending ACK 1351
[loss] dropped outgoing ACK 1351 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1351 already stored, re-sending ACK 1351
[loss] dropped outgoing ACK 1352 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1352 already stored, re-sending ACK 1352
[loss] dropped outgoing ACK 1353 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1353 already stored, re-sending ACK 1353
[loss] dropped outgoing ACK 1358 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1358 already stored, re-sending ACK 1358
[loss] dropped outgoing ACK 1364 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1364 already stored, re-sending ACK 1364
[loss] dropped outgoing ACK 1373 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1373 already stored, re-sending ACK 1373
[loss] dropped outgoing ACK 1384 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1384 already stored, re-sending ACK 1384
[loss] dropped outgoing ACK 1388 to 192.168.17.34:63247
[dup] 192.168.17.34:63247: chunk 1388 already stored, re-sending ACK 1388
[+] 192.168.17.34:63247 -> short.pdf: 1395/1395 chunks received
[+] 192.168.17.34:63247: short.pdf assembled (1394433 bytes, 169.097 s, 321 duplicate chunk(s), 0 out-of-order)
[+] 192.168.17.34:63247: MD5 VERIFIED (sender=e51f761c5bbf6c9b92b3c8e82840ba57, local=e51f761c5bbf6c9b92b3c8e82840ba57)
[+] Saved short.pdf (1394433 bytes)
[+] 192.168.17.34:63247 session closed
```

20% loss — download (GET)

UDPFTP Client:

```

[rtx] timeout on chunk 1346, retry 1/10
[+] 1350/1395 chunks acknowledged
[rtx] timeout on chunk 1373, retry 1/10
[rtx] timeout on chunk 1374, retry 1/10
[rtx] timeout on chunk 1380, retry 1/10
[rtx] timeout on chunk 1384, retry 1/10
[rtx] timeout on chunk 1391, retry 1/10
[rtx] timeout on chunk 1392, retry 1/10
[+] 1395/1395 chunks acknowledged
[rtx] no verdict for DONE, retry 1/5
[rtx] no verdict for DONE, retry 2/5
[rtx] no verdict for DONE, retry 3/5
[rtx] no verdict for DONE, retry 4/5
[rtx] no verdict for DONE, retry 5/5
[=] PUT short.pdf | 1394433 bytes | 171.295 s | 323 retransmitted chunk(s) | 7.95 KB/s | NO VERDICT
udp-ftp> █

```

UDFPTP Server:

The screenshot shows the Visual Studio Code editor with the file `UDFPTPServer.py` open. The file contains a `handle_control` function that manages file uploads and retries. The terminal output shows the server's log, including file storage paths, chunk acknowledgments, and the final assembly of the file `short.pdf`.

```

def handle_control(transport, sessions, sessions_lock, packet, address):
    receive_file, (filename, filesize, total_chunks)

    if command == "GET":
        if len(fields) < 2:
            transport.send_control("ERROR|malformed GET", address)
            return
        filename = fields[1]
        path = safe_storage_path(filename)
        if path is None or not os.path.isfile(path):

```

```

[dup] 192.168.17.34:64005: chunk 1310 already stored, re-sending ACK 1310
[loss] dropped outgoing ACK 1311 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1311 already stored, re-sending ACK 1311
[loss] dropped outgoing ACK 1319 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1319 already stored, re-sending ACK 1319
[loss] dropped outgoing ACK 1320 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1320 already stored, re-sending ACK 1320
[loss] dropped outgoing ACK 1321 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1321 already stored, re-sending ACK 1321
[loss] dropped outgoing ACK 1321 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1321 already stored, re-sending ACK 1321
[loss] dropped outgoing ACK 1326 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1326 already stored, re-sending ACK 1326
[loss] dropped outgoing ACK 1346 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1346 already stored, re-sending ACK 1346
[+] 192.168.17.34:64005 -> short.pdf: 1350/1395 chunks received
[loss] dropped outgoing ACK 1373 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1373 already stored, re-sending ACK 1373
[loss] dropped outgoing ACK 1374 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1374 already stored, re-sending ACK 1374
[loss] dropped outgoing ACK 1380 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1380 already stored, re-sending ACK 1380
[loss] dropped outgoing ACK 1384 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1384 already stored, re-sending ACK 1384
[loss] dropped outgoing ACK 1391 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1391 already stored, re-sending ACK 1391
[loss] dropped outgoing ACK 1392 to 192.168.17.34:64005
[dup] 192.168.17.34:64005: chunk 1392 already stored, re-sending ACK 1392
[+] 192.168.17.34:64005 -> short.pdf: 1395/1395 chunks received
[+] 192.168.17.34:64005: short.pdf assembled (1394433 bytes, 171.289 s, 323 duplicate chunk(s), 0 out-of-order)
[loss] dropped outgoing VERIFIED to 192.168.17.34:64005
[+] 192.168.17.34:64005: MD5 VERIFIED (sender=e51f761c5bbf6c9b92b3c8e82840ba57, local=e51f761c5bbf6c9b92b3c8e82840ba57)
[+] Saved short.pdf (1394433 bytes)

```

Loss vs. Performance Summary

The numbers reported by the client at the end of each run are collected below, for both directions of transfer and all three loss levels tested.

Request	File (bytes)	Loss	Time (s)	Retransmitted chunks	Throughput (KB/s)
PUT	short.pdf (1,394,433)	0%	3.88	0	350.86
PUT	short.pdf (1,394,433)	10%	80.22	140	16.97
PUT	short.pdf (1,394,433)	20%	169.10	321	8.05
GET	ACN.pdf (1,394,433)	0%	6.53	0	208.54
GET	ACN.pdf (1,394,433)	10%	90.35	165	15.07
GET	ACN.pdf (1,394,433)	20%	171.30	323	7.95

What the numbers show

Loss and performance move in lockstep here. Going from 0% to 10% loss already pushes retransmissions from zero up into the hundreds, and transfer time jumps by more than 20 times. Doubling the loss again to 20% roughly doubles both the retransmission count and the time taken, while throughput keeps falling — from the hundreds of KB/s at 0% loss down to single-digit KB/s at 20%. In short: every dropped packet costs a timeout-and-resend cycle, and those cycles add up fast once loss gets past a few percent.

Work Distribution

The work below was split between the two of us as follows.

Partner 1: GAJJALA JYOTHI SAI HARI TEJA — CS26MTECH11006

Task	Contribution	Challenges faced
Design & setup	Set up the TCP server skeleton and the UDP server skeleton, including socket binding and the request-handling loop for both.	Getting the framing right so messages didn't get mixed up on the socket stream.
TCP server-side integrity	Implemented the server-side SHA-256 checksum handling and the storage logic for incoming files.	Making sure the hash was computed after the file was fully flushed to disk.
Multi-client TCP server	Added threading on the TCP server so multiple clients could connect at once.	A race condition the first time two clients wrote to the log at the same moment.
UDP server reliability	Implemented the server-side chunking, acknowledgement sending, and reassembly logic for the UDP server.	Handling duplicate and out-of-order chunk arrivals correctly during reassembly.
Testing (server side)	Ran and screenshotted the server-side logs for the small-file, large-file, and multi-client scenarios on both TCP and UDP.	None major.

Partner 2: SAI CHAITANYA MUPPARAJU— CS26MTECH11008

Task	Contribution	Challenges faced
Design & setup	Set up the TCP client and UDP client skeletons, and chose a chunk size that balanced overhead against packet-loss risk.	Choosing a chunk size that balanced overhead against packet-loss risk.
TCP client integrity checks	Added client-side SHA-256 checksum verification and the auto-rename logic for existing files.	Making sure the hash was computed after the file was fully flushed to disk.
UDP client reliability layer	Implemented the client-side stop-and-wait acknowledgement and retry logic with a retry cap.	An early bug caused an infinite retry loop when a duplicate ACK arrived.
Loss simulation	Built the artificial packet/ACK drop feature used for the 10% and 20% loss tests.	Keeping the drop simulation deterministic enough to reproduce results.
Testing (client side)	Ran all six loss-rate experiments from the client side and put together the comparison table.	Long run times at 20% loss made repeated testing slow.

LLM USAGE:

```
PS C:\Users\saich\Desktop\Socket Programming-2> python
TCPFTPclient.py 10.120.202.116 10008
Connected to Mini-FTP Server. Commands: put <file>, get
<file>, list, quit
```

This issue usually happens because your TCP socket is hanging on a blocking `recv()` call or because the server closed the connection right after handling the file transfer. Even though the prompt `>` printed, the execution loop is stuck waiting for socket activity or stream sync.

1. The Server Closed the Connection Early

A common bug in custom FTP assignments is closing the client socket (`conn.close()`) immediately after completing the `put` command instead of keeping it open in a loop.

- Fix: Ensure your server wraps command handling in a persistent loop and only closes the

<https://gemini.google.com/app/6684c1dba3fb918b?hl=en-IN>

<https://claude.ai/chat/f61dcb1c-11b1-4595-9b29-19f760d7c652>

<https://chatgpt.com/share/6a95aa88-0148-83e8-80c0-7bedb6c0ed78>

UDP FTP client file download behavior ▾

```
python

target = unique_path(os.path.basename(filename))
if target != os.path.basename(filename):
    print("[*] %s already exists locally; saving as %s"
          % (filename, target))
```

`unique_path()` checks `os.path.exists(path)`, and if the file is already there, it doesn't stop and ask — it just appends " (1)", " (2)", etc., until it finds a name that's free, and returns that. The `print` right after is only informational ("hey, I renamed it"), not a prompt — there's no `input()` call anywhere in this path asking the user to confirm overwrite vs. rename.

So the behavior is by design: it guarantees you never lose an existing file, but it does so by renaming the incoming download rather than prompting.

If you want an actual "overwrite or save-as" prompt instead of silent renaming, you'd change that block to something like:

ANTI-PLAGIARISM STATEMENT:

We certify that this assignment/report is the result of our collaborative work, based on our collective study and research. All sources, including books, articles, software, datasets, reports, and communications, have been properly acknowledged. This work has not been previously submitted for assessment in any other course unless specific permission was granted by all involved instructors. We also acknowledge the use of AI tools, such as LLMs (e.g., ChatGPT, Gemini), for assistance in refining this assignment, if used. We have ensured that their usage complies with the academic integrity policies of this course. We pledge to uphold the principles of honesty, integrity, and responsibility at CSE@IITH. Additionally, we understand our duty to report any violations of academic integrity by others if we become aware of them.

Names <Roll Nos>:

SAI CHAITANYA MUPPARAJU<CS26MTECH11008>

GAJJALA JYOTHI SAI HARI TEJA<CS26MTECH11006>

Date : 31-08-2026

Signatures : Sai Chaitanya

Hari Teja